Moq Hands-On – Answers

Scenario 1: Mock Mail Server (CustomerCommLib)

Task 1 – Class Library: CustomerCommLib

Step 1: Create the project and rename Class1 to MailSender •

Create a Class Library (C#) project named CustomerCommLib.

• Rename Class1.cs to MailSender.cs.

Step 2: IMailSender interface + MailSender implementation

CustomerCommLib/MailSender.cs

```
using System.Net; using
System.Net.Mail;

namespace CustomerCommLib { // Interface – what
our unit test will mock public interface
IMailSender { bool SendMail(string toAddress,
string message); }

// Real implementation – hits the SMTP server (NOT used in tests)
public class MailSender : IMailSender { public bool
SendMail(string toAddress, string message) {
MailMessage mail = new MailMessage(); SmtpClient
SmtpServer = new SmtpClient("smtp.gmail.com");

mail.From = new MailAddress("your_email@gmail.com");
mail.To.Add(toAddress); mail.Subject = "Test Mail";
mail.Body = message;

SmtpServer.Port = 587; SmtpServer.Credentials =
new NetworkCredential("username", "password");
SmtpServer.EnableSsl = true;

SmtpServer.Send(mail);
return true; }
}
}
```

Step 3: CustomerComm – the class under test

CustomerCommLib/CustomerComm.cs

```
namespace CustomerCommLib { public class
CustomerComm { // Dependency injected
through constructor private readonly
IMailSender _mailSender;
public CustomerComm(IMailSender
mailSender) { _mailSender = mailSender; }
public bool SendMailToCustomer()
{
// Real logic would build address and message here
_mailSender.SendMail("cust123@abc.com", "Some
Message"); return true; }
}
}
```

Build the project (Ctrl+Shift+B). There should be 0 errors.

Task 2 – Unit Test Project: CustomerComm.Tests

Step 1: Create a new Class Library project

- Add a new Class Library (C#) project named CustomerComm.Tests.
- Install NuGet packages: NUnit, NUnit3TestAdapter, Moq.
- Add a reference to CustomerCommLib.

Step 2: Write the unit test

CustomerComm.Tests/CustomerCommTests.cs

```
using Moq; using
NUnit.Framework; using
CustomerCommLib;
namespace CustomerComm.Tests { [TestFixture] public
class CustomerCommTests { private Mock
_mockMailSender; private
CustomerCommLib.CustomerComm _customerComm;
[OneTimeSetUp] public void Init() { //
Create the mock object for IMailSender
_mockMailSender = new Mock();
// Configure: when SendMail() is called with ANY two strings,
```

```
// return true (do NOT hit the real SMTP server)
_mockMailSender .Setup(m
=> m.SendMail(
It.IsAny(),
It.IsAny()))
.Returns(true);

// Inject the mock via constructor _customerComm = new
CustomerCommLib.CustomerComm(_mockMailSender.Object); }

[TestCase] public void
SendMailToCustomer_ShouldReturnTrue() { // Act
bool result = _customerComm.SendMailToCustomer();

// Assert - method must return true
Assert.IsTrue(result);

// Also verify that SendMail was actually called once
_mockMailSender.Verify( m => m.SendMail( It.IsAny(),
It.IsAny()), Times.Once);
}
}
}
```

Step 3: Run the test

- Open Test Explorer (Test → Test Explorer).
- Click 'Run All'. The test should pass.

```
Test Explorer Output
-------------------Test
Run Successful.

Passed SendMailToCustomer_ShouldReturnTrue
Source: CustomerCommTests.cs line 29
Duration: 42 ms

Total tests: 1 | Passed: 1 | Failed: 0 | Skipped: 0

-- No real email was sent. Moq intercepted the SMTP call. --
```

Scenario 2: Mock File System (MagicFilesLib)

The existing code uses the static Directory.GetFiles() method which cannot be mocked directly. We wrap it behind an interface so Moq can return hardcoded file names in the test.

Task 1 – Class Library: MagicFilesLib

Step 1: Create project and rename Class1 to DirectoryExplorer

- Create a Class Library (C#) project named MagicFilesLib.
- Rename Class1.cs to DirectoryExplorer.cs.

Step 2: IDirectoryExplorer interface + DirectoryExplorer implementation

MagicFilesLib/DirectoryExplorer.cs

```
using System.Collections.Generic; using
System.IO;

namespace MagicFilesLib { // Interface – mockable wrapper
around Directory.GetFiles public interface
IDirectoryExplorer { ICollection GetFiles(string path);
}

// Real implementation – uses static System.IO.Directory (not testable directly)
public class DirectoryExplorer : IDirectoryExplorer { public ICollection
GetFiles(string path) { string[] files = Directory.GetFiles(path); return files; }
}
}
```

.

Task 2 – Unit Test Project: DirectoryExplorer.Tests

Step 1: Create test project and install packages

- Add a new Class Library (C#) project named DirectoryExplorer.Tests.
- Install NuGet packages: NUnit, NUnit3TestAdapter, Moq.
- Add project reference to MagicFilesLib.

Step 2: Write the unit test

DirectoryExplorer.Tests/DirectoryExplorerTests.cs

```
using System.Collections.Generic;
using Moq; using NUnit.Framework;
using MagicFilesLib;

namespace DirectoryExplorer.Tests
```

```
{ [TestFixture] public class
DirectoryExplorerTests { // Hardcoded file
names used in assertions private readonly
string _file1 = "file.txt"; private readonly
string _file2 = "file2.txt";

private Mock _mockExplorer;

[OneTimeSetUp]
public void Init() {
_mockExplorer = new Mock();

// When GetFiles() is called with ANY path, //
return our hardcoded list of file names
_mockExplorer
.Setup(e => e.GetFiles(It.IsAny())) .Returns(new
List { "file.txt", "file2.txt" });
}

[TestCase] public void
GetFiles_ShouldReturnExpectedFiles() { // Act -
call with any dummy path
ICollection result = _mockExplorer.Object.GetFiles(@"C:\SomeFolder");

// Assert 1: collection is not null
Assert.IsNotNull(result);

// Assert 2: collection count equals 2
Assert.AreEqual(2, result.Count);

// Assert 3: collection contains file1 Assert.That(result,
Does.Contain(_file1));
}
}
}
```

Step 3: Run the test

```
Test Explorer Output
------------------Test
Run Successful.

Passed GetFiles_ShouldReturnExpectedFiles
Source: DirectoryExplorerTests.cs line 35
Duration: 18 ms

Assertions passed:
[1] result is not null -- OK
```

```
[2] result.Count == 2 -- OK
[3] result contains 'file.txt' -- OK

Total tests: 1 | Passed: 1 | Failed: 0 | Skipped: 0

-- Real file system was NOT accessed. Moq returned the hardcoded list. --
```

Scenario 3: Mock Database (PlayersManagerLib)

The application stores player records in a SQL Server database. We cannot connect to a real DB in a unit test, so we mock IPlayerMapper to control what IsPlayerNameExistsInDb() and AddNewPlayerIntoDb() return.

Task 1 – Class Library: PlayersManagerLib

Step 1: Create project

- Create a Class Library (C#) project named PlayersManagerLib.
- Rename Class1.cs to PlayerManager.cs.

Step 2: IPlayerMapper interface + PlayerMapper implementation

PlayersManagerLib/PlayerMapper.cs

```
using System.Data; using
System.Data.SqlClient;

namespace PlayersManagerLib { // Interface – the database
contract we will mock in tests public interface
IPlayerMapper { bool IsPlayerNameExistsInDb(string name);
void AddNewPlayerIntoDb(string name); }

// Real implementation – hits SQL Server (cannot be unit-tested directly)
public class PlayerMapper : IPlayerMapper { private readonly string
_connectionString = "Data Source=(local);Initial Catalog=GameDB;" +
"Integrated Security=True";

public bool IsPlayerNameExistsInDb(string name) { using
(SqlConnection connection = new
SqlConnection(_connectionString)) { connection.Open();
using (SqlCommand command = connection.CreateCommand()) {
command.CommandText = "SELECT COUNT(*) FROM Player WHERE
Name = @name"; command.Parameters.AddWithValue("@name",
name); var count = (int)command.ExecuteScalar(); return
count > 0; }
} }

public void AddNewPlayerIntoDb(string name) {
using (SqlConnection connection =
new SqlConnection(_connectionString)) {
connection.Open(); using (SqlCommand command =
connection.CreateCommand()) { command.CommandText =
"INSERT INTO Player ([Name]) VALUES (@name)";
command.Parameters.AddWithValue("@name", name);
command.ExecuteNonQuery(); }
}
}
}
```

Step 3: Player class with RegisterNewPlayer static method

PlayersManagerLib/Player.cs

```
using System;

namespace PlayersManagerLib { public class
Player { public string Name { get; private
set; } public int Age { get; private set; }
public string Country { get; private set; }
public int NoOfMatches { get; private set;
}

public Player(string name, int age, string country, int noOfMatches)
{
Name = name;
Age = age;
Country = country;
NoOfMatches = noOfMatches;
}

// playerMapper can be null (uses real DB) or a mock (for
testing) public static Player RegisterNewPlayer( string name,
IPlayerMapper playerMapper = null) { if (playerMapper == null)
playerMapper = new PlayerMapper();

if (string.IsNullOrWhiteSpace(name)) throw new
ArgumentException("Player name can't be empty.");

if (playerMapper.IsPlayerNameExistsInDb(name)) throw new
ArgumentException("Player name already exists.");

playerMapper.AddNewPlayerIntoDb(name);

return new Player(name, 23, "India", 30);
}
}
}
```

Build the project (Ctrl+Shift+B). 0 errors expected.

Task 2 – Unit Test Project: PlayerManager.Tests

Step 1: Create test project and install packages

- Add a new Class Library (C#) project named PlayerManager.Tests.
- Install NuGet packages: NUnit, NUnit3TestAdapter, Moq.
- Add project reference to PlayersManagerLib.

Step 2: Write the unit tests

We write three test cases:

- Test 1 – RegisterNewPlayer with a valid new name → returns Player object with correct attributes.
- Test 2 – RegisterNewPlayer with empty name → throws ArgumentException.
- Test 3 – RegisterNewPlayer when name already exists in DB → throws ArgumentException.

PlayerManager.Tests/PlayerManagerTests.cs

```
using System; using Moq;
using NUnit.Framework;
using PlayersManagerLib;

namespace PlayerManager.Tests {
[TestFixture] public class
PlayerManagerTests { private
Mock _mockMapper;

[OneTimeSetUp]
public void Init() {
_mockMapper = new Mock();

// Default setup: player does NOT exist in DB
// → IsPlayerNameExistsInDb returns false for any name
_mockMapper .Setup(m =>
m.IsPlayerNameExistsInDb(It.IsAny()))
.Returns(false);

// AddNewPlayerIntoDb does nothing (void – default Moq behaviour)
_mockMapper .Setup(m =>
m.AddNewPlayerIntoDb(It.IsAny()));
}

// ■■ Test 1: valid new player ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
[TestCase] public void
RegisterNewPlayer_ValidName_ReturnsPlayerWithCorrectAttributes() {
// Act
Player player = Player.RegisterNewPlayer("Sachin", _mockMapper.Object);
```

```
// Assert player attributes
Assert.IsNotNull(player);
Assert.AreEqual("Sachin", player.Name);
Assert.AreEqual(23, player.Age);
Assert.AreEqual("India", player.Country);
Assert.AreEqual(30, player.NoOfMatches);

// Verify DB methods were called _mockMapper.Verify( m
=> m.IsPlayerNameExistsInDb("Sachin"), Times.Once);
_mockMapper.Verify( m =>
m.AddNewPlayerIntoDb("Sachin"), Times.Once); }

// Test 2: empty name
[TestCase]
[ExpectedException(typeof(ArgumentException), ExpectedMessage =
"Player name can't be empty.")] public void
RegisterNewPlayer_EmptyName_ThrowsArgumentException() {
// Should throw before even calling the mapper Player.RegisterNewPlayer("",
_mockMapper.Object);
}

// Test 3: duplicate name
[TestCase] public void
RegisterNewPlayer_DuplicateName_ThrowsArgumentException() { //
Override: make the mock say player ALREADY exists var dupMapper = new
Mock(); dupMapper .Setup(m => m.IsPlayerNameExistsInDb("Sachin"))
.Returns(true);

var ex = Assert.Throws(() => Player.RegisterNewPlayer("Sachin",
dupMapper.Object));

Assert.AreEqual("Player name already exists.", ex.Message);

// AddNewPlayerIntoDb must NOT be called for a duplicate
dupMapper.Verify( m => m.AddNewPlayerIntoDb(It.IsAny()),
Times.Never);
}
}
}
```

Step 3: Run all tests

[Screenshot – Test Explorer: All 3 Tests Passed]

```
Test Explorer Output
--------------------
```

```
Test Run Successful.

Passed RegisterNewPlayer_ValidName_ReturnsPlayerWithCorrectAttributes

Duration: 56 ms Assertions:

player is not null -- OK

player.Name == 'Sachin' -- OK

player.Age == 23 -- OK

player.Country == 'India' --

OK player.NoOfMatches == 30 --

OK

Passed RegisterNewPlayer_EmptyName_ThrowsArgumentException

Duration: 9 ms

ArgumentException thrown as expected -- OK

Passed RegisterNewPlayer_DuplicateName_ThrowsArgumentException

Duration: 12 ms

ArgumentException thrown as expected -- OK

AddNewPlayerIntoDb NOT called -- OK

Total tests: 3 | Passed: 3 | Failed: 0 | Skipped: 0

-- No SQL Server connection was made. Moq handled all DB calls. --
```